PyTorch Cheatsheet

Tensors • Autograd • Neural Nets • Training — at a glance

Beginner-friendly | v2.x | GPU-ready | Fun!

◆ Tensor Creation

Tensors are the heart of PyTorch — n-dim arrays that live on CPU or GPU and track gradients.

```
import torch

# From data
x = torch.tensor([[1, 2], [3, 4]])

# Special tensors
z = torch.zeros(3, 4)
o = torch.ones(2, 5)
r = torch.randn(3, 3)   # normal
i = torch.eye(4)        # identity
a = torch.arange(0, 10, 2)
```

✦ Tensor Operations

Element-wise math, reshaping, indexing — all vectorized and lightning fast.

```
# Math
y = x + 2;  y = x * x
z = torch.matmul(a, b)  # or a @ b

# Shape
x.view(2, -1)     # reshape
x.unsqueeze(0)    # add dim
x.squeeze()       # drop size-1 dims
x.transpose(0, 1)
torch.cat([a, b], dim=0)
```

∂ Autograd Magic

PyTorch tracks every op on tensors with requires_grad=True. Call .backward() and gradients flow back automatically.

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3 + 2*x
y.backward()
print(x.grad)   # 3x^2 + 2 = 14

with torch.no_grad():
    eval_pred = model(x)
```

▲ Device & GPU

Move tensors and models to GPU with one line. Mixed devices? PyTorch will yell.

```
device = torch.device(
  'cuda' if torch.cuda.is_available()
  else 'cpu')

x = x.to(device)
model = model.to(device)
torch.cuda.empty_cache()
```

★ Quick tip

Use `torch.tensor(data)` to copy, `torch.as_tensor(data)` to share memory when possible.

Neural Networks

Build, train, and unleash your models

nn.Module · Loss · Optim · Train Loop

★ Defining a Model with nn.Module

Subclass nn.Module, register layers in __init__, define forward(). Done.

```
import torch.nn as nn, torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim=784, hidden=128, out_dim=10):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.fc2 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

● Loss Functions

Pick the loss to match your task.

```
# Classification
criterion = nn.CrossEntropyLoss()

# Regression
criterion = nn.MSELoss()

# Binary
criterion = nn.BCEWithLogitsLoss()

loss = criterion(pred, target)
```

✪ Optimizers

Adam is a great default. SGD+momentum wins for vision.

```
import torch.optim as optim

opt = optim.Adam(model.parameters(),
                 lr=1e-3)

# or
opt = optim.SGD(model.parameters(),
                lr=0.01, momentum=0.9)

scheduler = optim.lr_scheduler.StepLR(
    opt, step_size=10, gamma=0.5)
```

❖ The Sacred Training Loop

Five steps. Memorize them. They show up everywhere.

1 zero · 2 forward · 3 loss · 4 backward · 5 step

```
model.train()
for epoch in range(num_epochs):
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()       # 1  clear old grads
        pred = model(x)             # 2  forward pass
        loss = criterion(pred, y)   # 3  compute loss
        loss.backward()             # 4  backprop
        optimizer.step()            # 5  update weights
```

Data, Layers & Production

Feed it data, pick the right layers, ship it.

❖ Datasets & DataLoaders

Wrap your data, get batching, shuffling, and parallel loading for free.

```
from torch.utils.data import (
  Dataset, DataLoader)

class MyData(Dataset):
    def __len__(self): ...
    def __getitem__(self, idx): ...

loader = DataLoader(ds, batch_size=32,
    shuffle=True, num_workers=4)
```

★ Save & Load Models

Always save the state_dict — not the model object — for portability.

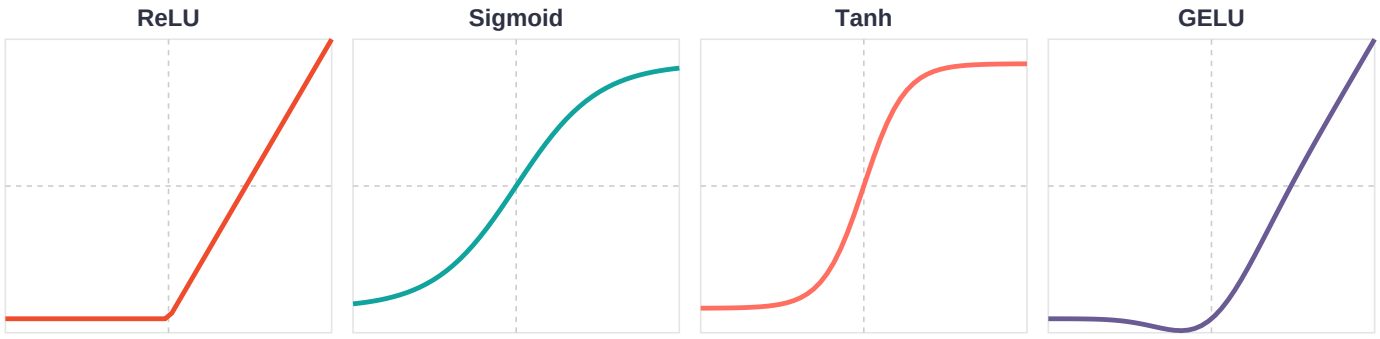
```
# Save
torch.save(model.state_dict(),
           'model.pt')

# Load
model = MLP()
model.load_state_dict(
  torch.load('model.pt'))
model.eval()
```

❖ Common Layers — Pick Your Weapon

Layer	Use Case	Quick Code
nn.Linear	Fully connected / MLP	`nn.Linear(in_f, out_f)`
nn.Conv2d	Image features	`nn.Conv2d(3, 16, kernel_size=3)`
nn.LSTM	Sequences, time series	`nn.LSTM(in_size, hidden, num_layers)`
nn.Embedding	Tokens → vectors	`nn.Embedding(vocab, dim)`
nn.BatchNorm2d	Stable & fast training	`nn.BatchNorm2d(num_features)`
nn.Dropout	Regularization	`nn.Dropout(p=0.5)`
nn.Transformer	Attention all the way	`nn.Transformer(d_model, nhead)`
nn.LayerNorm	Norm for transformers	`nn.LayerNorm(dim)`

▲ Activations — Visualized



Use F.relu / torch.sigmoid / torch.tanh / F.gelu • ReLU is the default workhorse, GELU rules transformers